SENAI • Inteligência Artificial Generativa

VISÃO COMPUTACIONAL APLICADA

Reconhecimento Facial com Python e OpenCV

Apostila Autodidata e Completa

Prof. Hebert Felix

Baseado no material: "aula_reconhecimento_facial_senai_40_slides_hebert_felix-1.pptx"

Capítulo 1: Preparação, Arquitetura e o Pipeline de IA

Antes de escrevermos a primeira linha de código, é crucial compreender que um sistema de Inteligência Artificial não é mágica; é um fluxo estruturado de dados (pipeline). O desenvolvimento de Visão Computacional exige um ambiente controlado e uma arquitetura de pastas bem definida.

1.1. O Ambiente e as Ferramentas (Stack)

O projeto foi desenhado para ser executado no **VS Code** utilizando **Python 3.12.9** [cite: 2]. Para garantir que as bibliotecas não entrem em conflito com outros projetos do seu computador, utilizamos um Ambiente Virtual (`venv`) [cite: 2].

> **POR QUE USAR AMBIENTE VIRTUAL (VENV)?**
>
> A biblioteca OpenCV possui dezenas de versões. O ambiente virtual (`python -m venv .venv`) isola o projeto atual [cite: 2], criando uma "bolha" segura onde apenas as dependências específicas desta aula existirão. No VS Code, selecione o interpretador dentro da pasta `.venv` [cite: 2].

A dependência central do nosso projeto é o **opencv-contrib-python** [cite: 2]. Muitos alunos cometem o erro de instalar o `opencv-python` tradicional, mas a versão "contrib" é necessária para usar o módulo `cv2.face`, fundamental para o nosso reconhecedor [cite: 2].

1.2. Detecção vs. Reconhecimento

Um erro conceitual grave no mercado é confundir detecção com reconhecimento. São etapas diferentes e complementares [cite: 2]:

- **Detecção Facial (Haar Cascade):** Responde à pergunta *"existe um rosto na imagem?"* [cite: 2]. A saída são as coordenadas x, y, largura e altura [cite: 2]. Ele não sabe quem é a pessoa [cite: 2].
- **Reconhecimento Facial (LBPH):** Responde à pergunta *"de quem é esse rosto?"* [cite: 2]. Ele compara o rosto com o padrão treinado e a saída pode ser o ID, o nome da pessoa ou a indicação de que é um "desconhecido" [cite: 2].

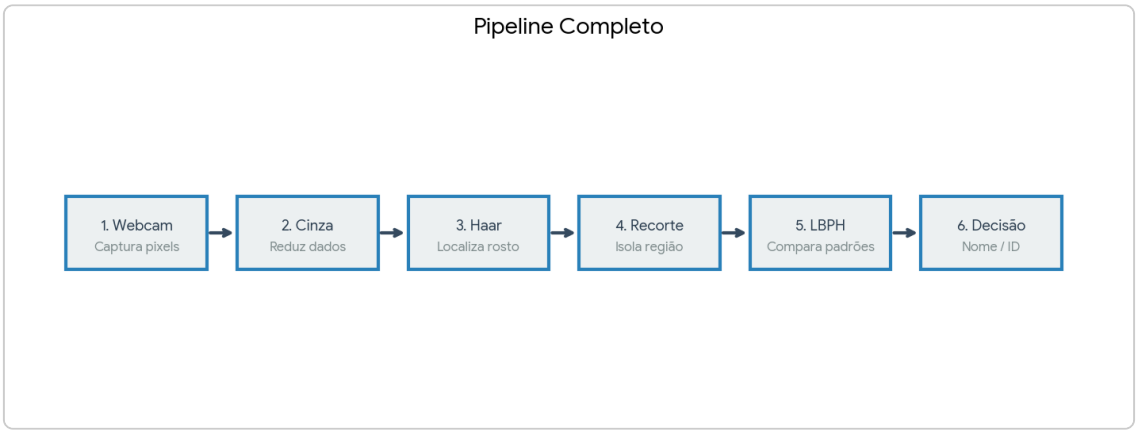


Figura 1: O mapa mental do pipeline completo, da captura do pixel à decisão biométrica (Conforme slide 32).

Capítulo 2: Script de Cadastro (cadastrar_rosto.py)

A primeira etapa de qualquer modelo de IA é a coleta de dados de qualidade. Nosso objetivo é capturar 20 imagens do rosto usando a webcam [cite: 2].

2.1. O Motor de Busca: Cascade Classifier

```
detector = cv2.CascadeClassifier(
    cv2.data.haarcascades + "haarcascade_frontalface_default.xml"
)
```

O `CascadeClassifier` carrega um classificador pré-treinado em formato XML [cite: 2]. O OpenCV já vem com o arquivo `haarcascade_frontalface_default.xml`, cujo caminho interno é acessado por `cv2.data.haarcascades` [cite: 2]. O Haar Cascade atua como o detector que localiza regiões candidatas a rosto na imagem [cite: 2].

2.2. A Matemática da Escala de Cinza

```
cinza = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

O frame original capturado da webcam possui três canais (BGR) [cite: 2]. O método `cv2.cvtColor` com o código `cv2.COLOR_BGR2GRAY` converte esse padrão para tons de cinza [cite: 2]. Processar a imagem em cinza significa trabalhar com menos dados e de forma mais simples, o que é adequado tanto para o Haar Cascade quanto para o LBPH [cite: 2].

2.3. Parâmetros de Detecção (detectMultiScale)

```
rostos = detector.detectMultiScale(cinza, 1.1, 5)
```

A função `detectMultiScale` localiza os rostos e devolve os retângulos onde eles foram encontrados [cite: 2]. Entendendo os parâmetros:

- **`cinza`:** É a imagem que está sendo analisada pelo detector [cite: 2].
- **`1.1` (scaleFactor):** O algoritmo reduz a escala de busca em 10% a cada tentativa [cite: 2].
- **`5` (minNeighbors):** Define que o algoritmo exige confirmações para aceitar que encontrou um rosto [cite: 2].

2.4. A Geometria do Recorte: x, y, w, h

Quando o rosto é detectado, a saída é uma lista de retângulos, definidos pelas variáveis `x, y, w, h` [cite: 2].

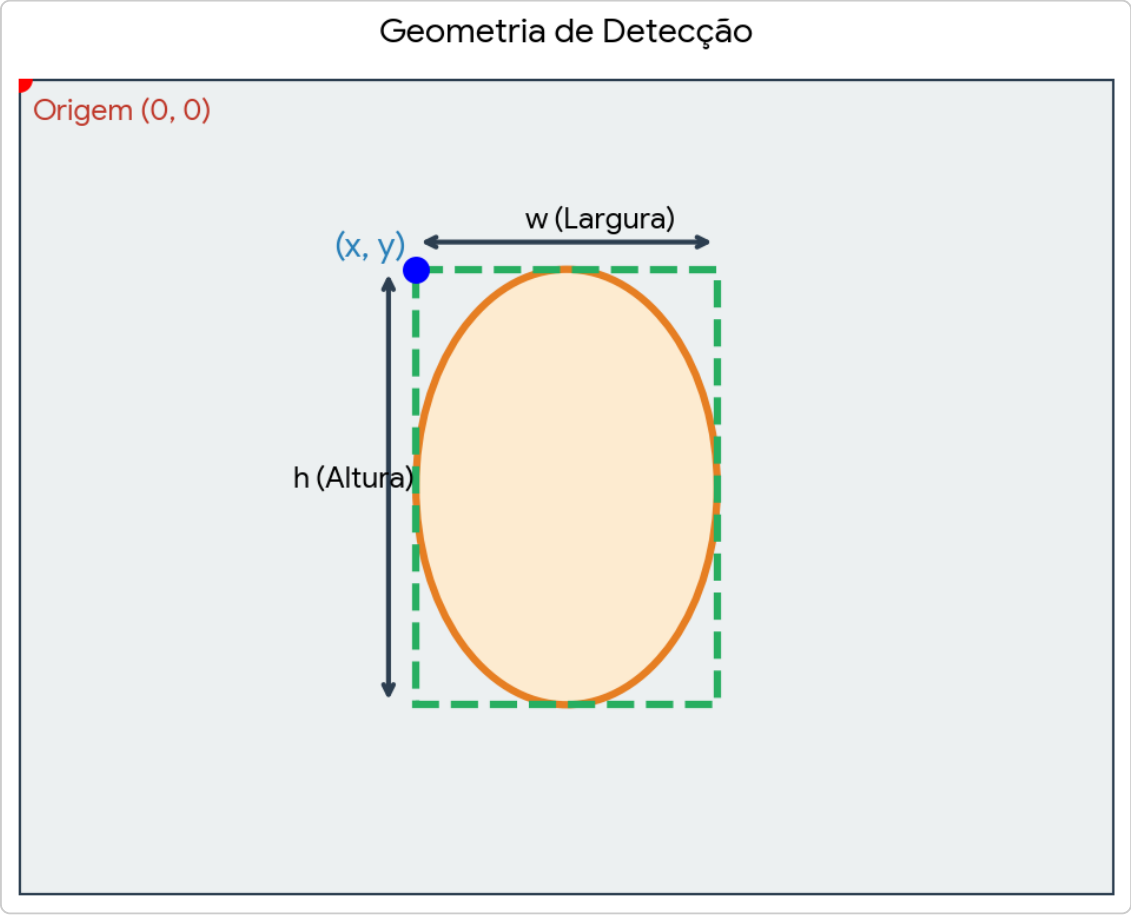


Figura 2: A anatomia do retângulo do rosto no sistema de coordenadas (Conforme slide 34).

- **`x` e `y`:** Representam a posição horizontal e vertical iniciais do retângulo [cite: 2].
- **`w` e `h`:** Representam a largura e a altura do rosto [cite: 2].

Para recortar a região do rosto, o OpenCV usa a notação: `cinza[y:y+h, x:x+w]` [cite: 2]. Após o recorte, a imagem é padronizada para 200 x 200 pixels usando `cv2.resize()`, pois o reconhecedor precisa que as imagens de teste e treinamento tenham um tamanho semelhante [cite: 2].

Capítulo 3: Script de Reconhecimento (reconhecer_rosto.py)

O segundo script carrega as imagens, treina o reconhecedor LBPH e usa a webcam para comparar o rosto em tempo real e exibir o nome [cite: 2].

3.1. O Algoritmo LBPH (Local Binary Patterns Histograms)

```
reconhecedor = cv2.face.LBPHFaceRecognizer_create()
```

O `LBPHFaceRecognizer_create()` cria o modelo reconhecedor [cite: 2]. O LBPH funciona observando as diferenças entre pixels vizinhos para criar padrões binários locais, agrupando-os em histogramas que caracterizam o rosto [cite: 2].

3.2. Carga de Dados e Estruturação (O Treinamento)

```
imagens = []
ids = []

for arquivo in os.listdir("rostos"):
    imagem = cv2.imread("rostos/" + arquivo, cv2.IMREAD_GRAYSCALE)
    imagens.append(imagem)
    ids.append(1)  # ID 1 = Hebert

reconhecedor.train(imagens, np.array(ids))
```

O código lê todas as imagens da pasta `rostos` e as armazena na lista `imagens`, enquanto adiciona o ID 1 na lista `ids` [cite: 2]. É essencial entender que o modelo não entende nomes, ele entende números, e todas as fotos da pessoa devem ter o mesmo ID [cite: 2]. O método

`reconhecedor.train()` ensina o modelo usando as imagens e a lista de IDs, que é convertida para um array através da biblioteca NumPy (`np.array(ids)`) [cite: 2].

3.3. Inferência em Tempo Real e Limite de Confiança (Threshold)

Com o modelo treinado, capturamos a face com a webcam e tentamos reconhecê-la [cite: 2].

```
id, distancia = reconhecedor.predict(rosto)

if id == 1 and distancia < 70:
    nome = "Hebert"
else:
    nome = "Desconhecido"
```

O método `predict(rosto)` compara o rosto encontrado com o treinamento e retorna o identificador (ID) previsto e uma distância [cite: 2].

A REGRA DA DISTÂNCIA

A métrica retornada não é uma porcentagem [cite: 2]. É uma medida de diferença: uma distância menor significa que o padrão está mais próximo do treino, indicando maior semelhança [cite: 2]. Uma distância maior aponta menor semelhança [cite: 2]. O valor 70 que usamos no código (`distancia < 70`) é didático e deve ser ajustado conforme iluminação e dataset reais [cite: 2]. Se a distância estiver abaixo de 70, aceitamos como "Hebert" [cite: 2]. Evite falar em "70% de confiança" [cite: 2].

Capítulo 4: Prática, Limitações e Desafio Final

4.1. Limitações Técnicas e Prática Investigativa

Os modelos podem falhar dependendo das condições. A prática pede testes para entender os erros do sistema frente a variáveis como: iluminação insuficiente, rosto inclinado, uso de óculos, rosto distante, tentar reconhecer outra pessoa ou testar uma foto em uma tela [cite: 2]. Algumas limitações técnicas incluem poucas amostras, mudanças de aparência e falsos positivos/ negativos [cite: 2].

4.2. Ética e Dados Sensíveis

Projetos com rostos lidam com dados biométricos sensíveis e requerem ética [cite: 2]. Em um projeto real, é necessário pensar na finalidade, na necessidade e na segurança desses dados [cite: 2]. A boa prática recomenda que em sala de aula se use imagens autorizadas e remova os arquivos do sistema depois [cite: 2]. Importante ressaltar que IA Generativa, como LLMs, produz conteúdo novo, enquanto a visão clássica apenas analisa os dados [cite: 2].

4.3. Desafio Final (Missão SENAI)

O desafio final, proposto no slide 40, é escalar o projeto e transformar o reconhecedor de 1 ID para 2 IDs [cite: 2].

```
nomes = {
    1: "Hebert",
    2: "Aluno"
}
nome = nomes.get(id_pessoa, "Desconhecido")
```

Critérios: Você deverá cadastrar duas pessoas e associar cada uma a um ID distinto [cite: 2]. Em seguida, treinar o sistema com ambos os IDs e testar para exibir o nome correto [cite: 2]. E, por fim, testar também com o rosto de alguém não cadastrado [cite: 2].

Bons estudos e excelente codificação!